Build a Real-Time Text to Audio Converter Pipeline using Amazon Polly

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT

Under the guidance of

Mrs. Sumana Bethala, APSSDC

Mr. Juttuka.Lovababu, APSSDC

Submitted by

Perungulam Kanaka Laya Sadhana, 23BQ1A05H7, VVIT Guntur(Batch - 1),

Lalam Girishma, 23bq1a4294, VVIT Guntur(Batch - 1),

MADDURI POOJA, 24bq1a05R3,, VVIT Gunutr(Batch - 2),

MAKKE SAI LAVANYA, 24BQ1A05R9, VVIT Gunutr(Batch - 2),

Nunna Vaishnavi,25BQ5A0537, VVIT Gunutr(Batch - 2)

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of my summer internship project at **Andhra Pradesh Skill Development Corporation (APSSDC)**. This opportunity has been an enriching and transformative experience for me, and I am truly thankful for the support, guidance, and encouragement I have received along the way.

First and foremost, I extend my sincere regards to Mrs Sumana Bethala and Mr J Lovabu, my supervisors and mentors, for providing me with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

I would like to thank the entire team at **Andhra Pradesh Skill Development Corporation (APSSDC)** for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback I received from my colleagues significantly contributed to the development and success of this project.

In conclusion, I am honoured to have been a part of this internship program, and I look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.

Thank you.

Sincerely,

Perungulam Kanaka Laya Sadhana,23BQ1A05H7,layasadhana07@gmail.com.
Lalam Girishma, 23bq1a4294, 23bq1a4294@vvit.net.
Madduri Pooja, 24BQ1A05R3, madduri1975@gmail.com.
Makke Sai Lavanya, 24BQ1A05R9, lavanyamakke@gmail.com.
Nunna Vaishnavi, 25BQ5A0537, nunna042503@gmail.com.

ABSTRACT

This project presents the implementation of an **Event-Driven Text-to-Speech (TTS) Translation Pipeline** using l**Amazon Polly** to automate document synthesis and audio storage workflows. The primary goal is to leverage serverless computing to build an efficient, scalable, and event-driven solution that reduces infrastructure overhead while ensuring real-time responsiveness. The system workflow begins with the upload of a text file **(.txt)** into an input Amazon S3 bucket designated as **text-storage-bkt.** Upon this event, an AWS Lambda function named **pollytranslationfunction**, running on a Python 3.14 runtime environment, is automatically triggered via an **s3:ObjectCreated**:Put notification to parse the incoming file contents. The Lambda function processes the text data and interacts directly with Amazon Polly to synthesize the text into an audio format. To ensure system resilience and fault tolerance, an asynchronous invocation destination is configured to automatically route execution failures to an output S3 bucket named **audio-storage-bkt.**

This architecture is fully serverless, utilizing **AWS Lambda for core computation**, **Amazon S3** for source text and destination audio storage, **Amazon Polly for media generation**, and an AWS **Identity and Access Management (IAM)** role named **pollytranslationrole**—embedded with **AmazonPollyFullAccess, AmazonS3FullAccess**, and **AWSLambdaBasicExecutionRole** policies—for secure service permissions. The solution offers high scalability, reduced operational complexity, and faster response to data ingestion changes without any need for manual intervention or traditional server management.

TABLE OF CONTENTS

1. INTRODUCTION

In the digital era, where agility, automation, and real-time decision-making have become essential to business success, efficient inventory management has emerged as a critical operational need. This project, title **Build a Real – Time Text to Audio Converter Pipeline using Amazon Polly**, harnesses the power of **serverless computing on Amazon Web Services (AWS)** to deliver a robust, scalable, and fully automated solution for tracking and maintaining inventory. It stands as a testament to how cloud technologies can transform traditional resource-heavy processes into intelligent, event-driven workflows with minimal infrastructure overhead.

Modern enterprises and digital content platforms demand uninterrupted accessibility to information, scalable media distribution networks, and seamless data processing pipelines. Legacy text-to-speech architectures often fall short in scalability and flexibility, struggling under variable translation workloads, suffering from high inference latency, and requiring constant infrastructure maintenance. This project meets those challenges by adopting a fully serverless architecture—where AWS services work in concert to automate data ingestion, speech synthesis, and fault-tolerant routing, all while maintaining high availability and operational efficiency. Designed with simplicity, reliability, and automation at its core, the system allows users to upload a **textual document (`.txt`)** containing raw written content into an input Amazon S3 bucket designated as **`text-storage-bkt`**. This event automatically triggers an AWS Lambda function named **pollytranslationfunction** running on a **Python 3.14 environment**, which parses the file and securely communicates with Amazon Polly—a fast, highly scalable cloud service that uses advanced deep learning technologies to synthesize lifelike speech

2.METHODOLOGY

The development of the Event-Driven Text-to-Speech Translation Pipeline followed a structured and iterative methodology to ensure the successful fulfillment of project objectives. The methodology was divided into key phases, including requirements gathering, system design, implementation, testing, deployment, and refinement. Each phase was designed to align with best practices in cloud-native development and serverless architecture. The following subsections outline each phase in detail.

2.1 Requirements Gathering and Feasibility Study

The initial phase focused on defining the project scope and analyzing the computational prerequisites for automating media synthesis. The core functional requirement was to establish a fully automated pipeline capable of ingesting text assets and converting them into audible formats without manual intervention. Technical feasibility studies were conducted on AWS serverless services to determine optimal compute limits, trigger mechanisms, and language synthesis features, ensuring the system could reliably process data while keeping operational overhead low**.**

2.2 System and Security Architecture Design

During this phase, the decoupled cloud-native architecture was mapped out. The system topology was structured around an event-driven model where storage events act as direct compute triggers. Security design was finalized by constructing a custom AWS Identity and **Access Management (IAM)** role named **pollytranslationrole**. Adhering to the principle of least privilege, this role was pre-configured with granular security policies—specifically **AmazonPollyFullAccess**, **AmazonS3FullAccess**, and **AWSLambdaBasicExecutionRole**—to enable secure, isolated service-to-service communication.

2.3 Implementation and Serverless Development

The implementation phase translated the architectural design into working code. An AWS Lambda function named **pollytranslationfunction** was developed using the Python 3.14 runtime environment. The core script utilizes the **AWS SDK (Boto3) to catch S3 event buckets**, parse incoming raw textual data, and invoke Amazon Polly's synthesis engine. Two distinct Amazon S3 buckets were provisioned: **text-storage-bkt** to handle raw text file ingestion, and **audio-storage-bkt** to act as an asynchronous fallback destination for robust handling of operational payloads.

2.4 Event Integration and Trigger Configuration

This phase involved binding the independent serverless components into a synchronized pipeline. An event notification trigger was explicitly bound to **text-storage-bkt**, configured to listen for **s3:ObjectCreated**:Put events restricted to files with a **.txt** suffix. Additionally, asynchronous invocation destination rules were mapped to the Lambda function. This step established automated error routing, ensuring that if the execution lifecycle encountered an ingestion failure, the system would immediately preserve the state within the designated target storage without breaking the pipeline.

2.5 Testing and Validation

Rigorous validation testing was executed to evaluate the performance, latency, and fault tolerance of the automated workflow. Test cases included uploading standard format text files, empty files, and unsupported extensions to check trigger accuracy. CloudWatch logs were monitored to analyze the execution duration of the Python function and to verify that the speech generation engine delivered high-fidelity audio output. The asynchronous error handling mechanism was intentionally stressed to confirm that failed payloads were successfully isolated

and routed.

2.6 Deployment and Refinement

Upon successful validation, the finalized serverless configurations, resource bindings, and access controls were deployed directly onto the AWS global infrastructure. Post-deployment refinement focused on optimizing the Lambda execution timeout parameters and managing session durations to maximize cost-efficiency. This iterative refinement produced a highly responsive, scalable, and self-managed processing environment capable of handling production-grade textual synthesis dynamically**.**

3 TECHNOLOGY STACK

The Event-Driven Text-to-Speech Translation Pipeline was developed using a modern, cloud-native technology stack that supports event-driven architecture, real-time media synthesis, and scalability. The following technologies and services were used throughout the implementation:

3.3 Cloud Platform:

3.3.1 Amazon Web Services (AWS):

The entire project is hosted on AWS, leveraging its managed serverless ecosystem for compute, file storage, artificial intelligence capabilities, and secure access management.

3.4 AWS Services

AWS Lambda: Executes the core backend computation without server provisioning. The function `pollytranslationfunction` handles parsing the text objects, communicating with synthesis engines, and structuring the file workflows.

Amazon Polly: A cloud-native Artificial Intelligence service that converts text into life-like speech, acting as the primary synthesis engine for translating raw document data into high-fidelity audio streams.

3.4.1 AWS IAM (Identity and Access Management) : Manages identity and infrastructure security by defining the `pollytranslationrole`, ensuring authenticated execution vectors across the platform

3.4.2 Amazon CloudWatch : Provides centralized system monitoring, capturing performance metrics and hosting real-time runtime tracking logs.

3.5 Programming & Scripting

3.5.1 Python 3.14 (Lambda Runtime) : The high-level scripting language used to author the backend automation logic within the AWS Lambda environment, optimized for modern cloud execution boundaries.

3.5.2 Boto3 SDK: Python SDK for AWS used within Lambda to interact with S3, DynamoDB, and SNS.

3.6 Data Format

TXT (Plain Text Format): The structured input data format constraint configured using file suffixes (`.txt`) to isolate relevant upload events within the storage layer.

3.7 Security & Access

IAM Roles and Policies: Configured following the security principle of least privilege. The execution role specifically binds the following managed policies to isolate service actions:

AmazonPollyFullAccess : Grants permissions to synthesize speech from textual input.

AmazonS3FullAccess: Allows reading raw objects and writing output states to buckets.

AWSLambdaBasicExecutionRole: Authorizes the creation of CloudWatch logging streams.

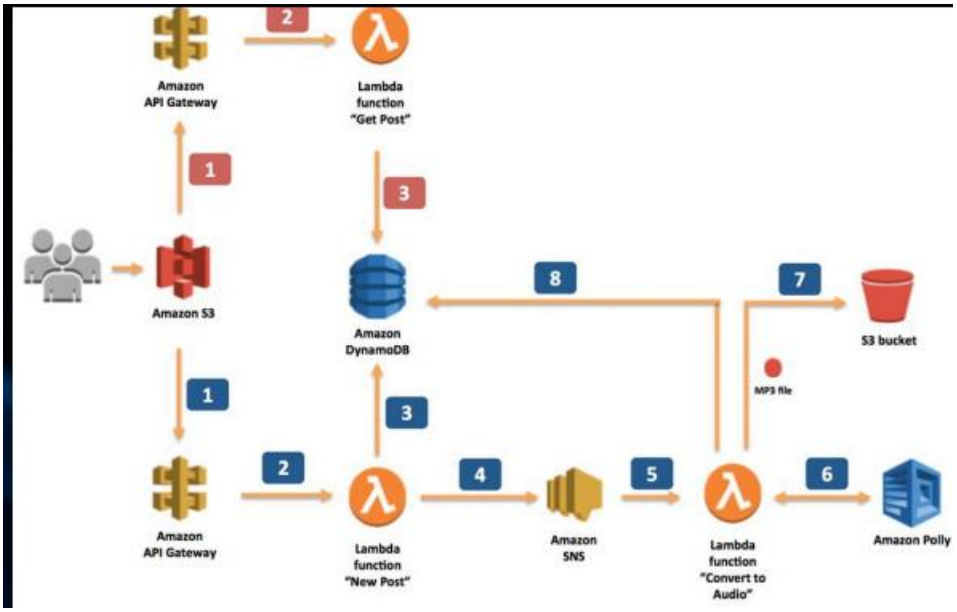
3.8 Monitoring & Logging:

3.8.1 CloudWatch Logs: Captures stdout/stderr execution streams generated during Lambda runtime invocations for precise debugging, error tracking, and payload tracing.

CloudWatch Metrics: Tracks operational performance dimensions including function invocation counts, runtime durations, throttling events, and execution error rates.

4 SYSTEM DESIGN / ARCHITECTURE

The Event-Driven Text-to-Speech Translation Pipeline is designed as a serverless, cloud-native solution that automates textual processing and high-fidelity speech synthesis. The architecture adopts a completely serverless model, ensuring automatic horizontal scaling, zero ongoing server maintenance, and a strict pay-per-use cost structure. The system follows a decoupled, multi-tier design comprising a data ingestion layer, a core processing compute tier, a speech synthesis engine, and an asynchronous fallback storage tier. All layers are tightly integrated using event-driven communication protocols and secured through managed AWS policies. This section outlines the core components and their interactions in the architecture.



4.3 Data Ingestion Layer – Amazon S3

The entry point of the pipeline is built entirely on Amazon S3 object storage via `text-storage-bkt`. This layer isolates storage functions from computational processing, allowing systems or users to upload source plain text files (`.txt`) at scale. The storage tier acts as an event producer;

it is continuously monitored by a native S3 Event Notification filter that detects `s3:ObjectCreated:Put` actions. This targeted filtering ensures that compute resources are invoked exclusively when a valid textual artifact enters the pipeline.

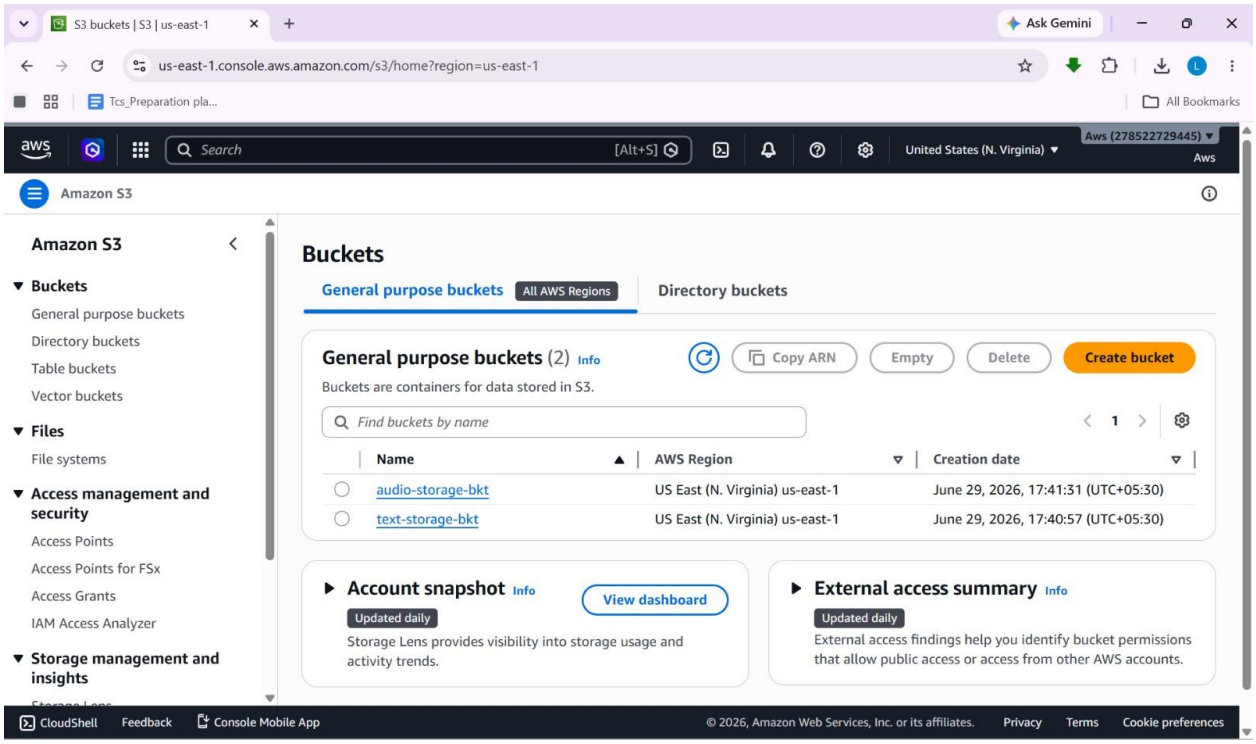
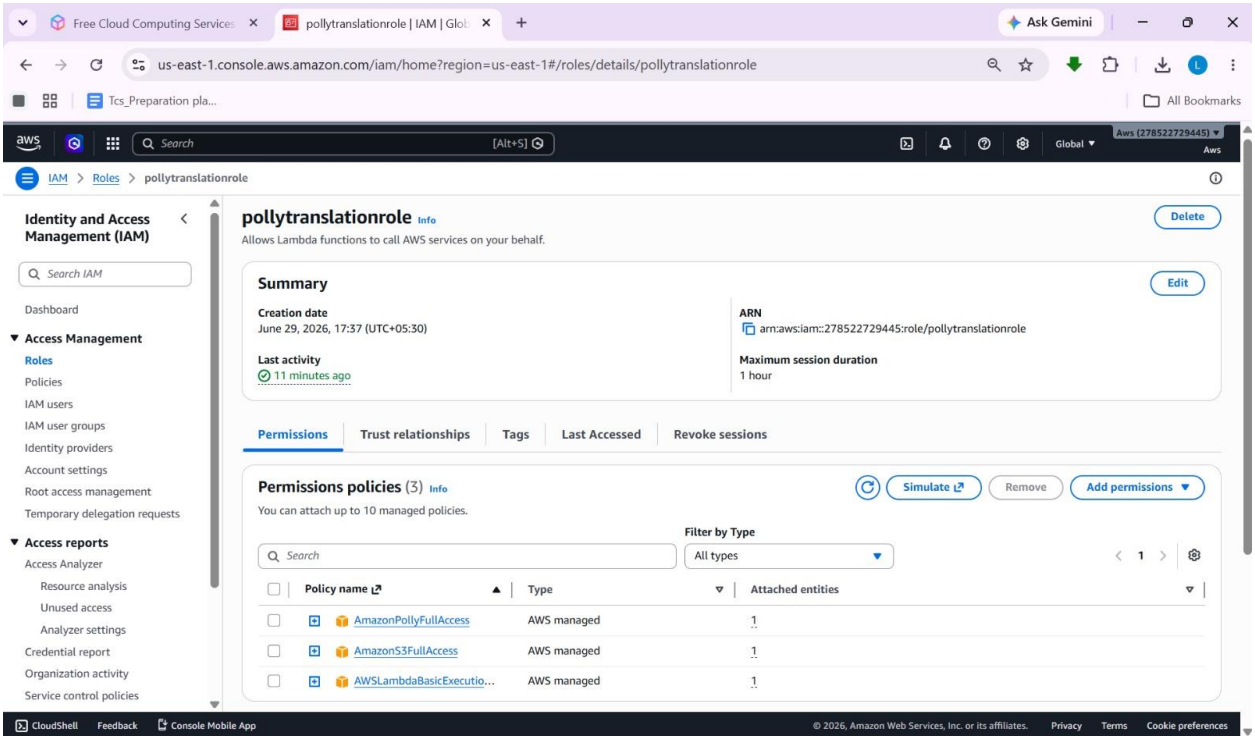


Fig 1: Buckets Creation for entry and exit points

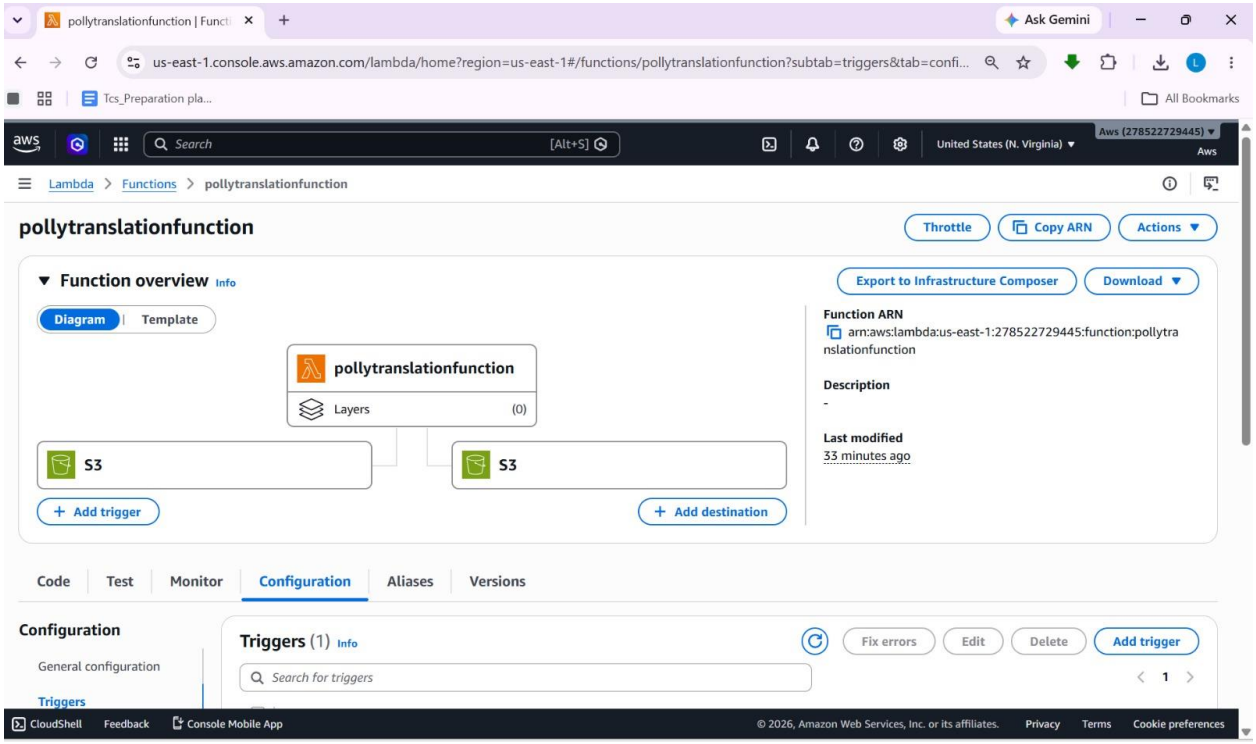
4.2 Compute and Processing Logic Tier:

The core coordination of the pipeline is managed by AWS Lambda through the function `pollytranslationfunction`. Operating on a Python 3.14 runtime environment, this compute engine is natively invoked by the ingestion layer's event signals. Because it is serverless, the execution framework spins up matching compute instances dynamically in response to workload volume. The function extracts context from the JSON event payload, downloads the target textual object using the Boto3 SDK, and prepares the data blocks for stream translation.



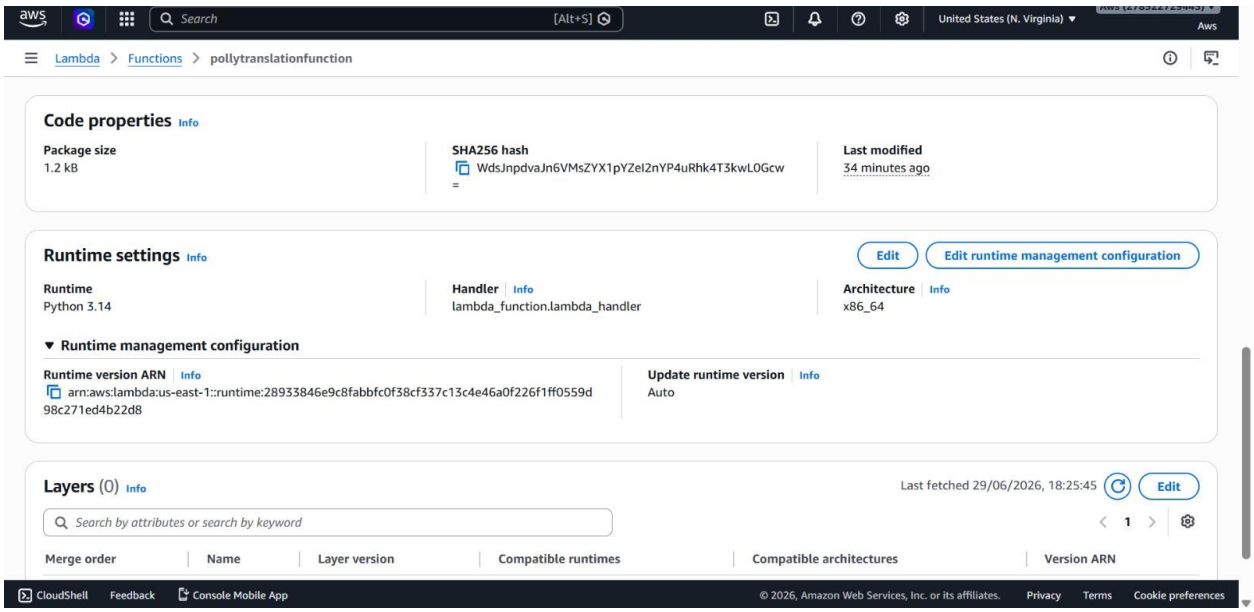
4.3 Speech Synthesis Engine:

The conversion tier leverages Amazon Polly to handle artificial intelligence speech synthesis. Instead of employing complex, local deep learning dependencies, the Lambda function relies on Polly's managed API. The processing tier passes the extracted text blocks to Amazon Polly, specifying voice parameters and synthesis features. Polly dynamically processes the characters into natural-sounding speech, outputting a clear audio stream back to the calling script or direct delivery channels.



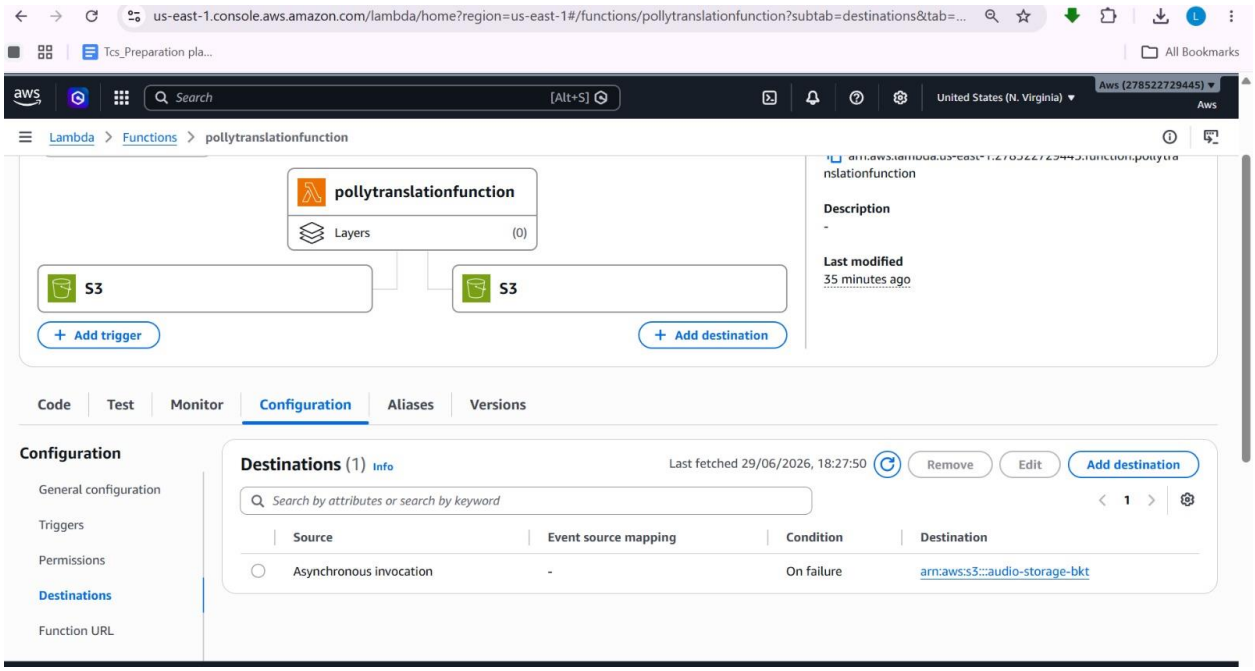
4.4 Security and Access Control Context:

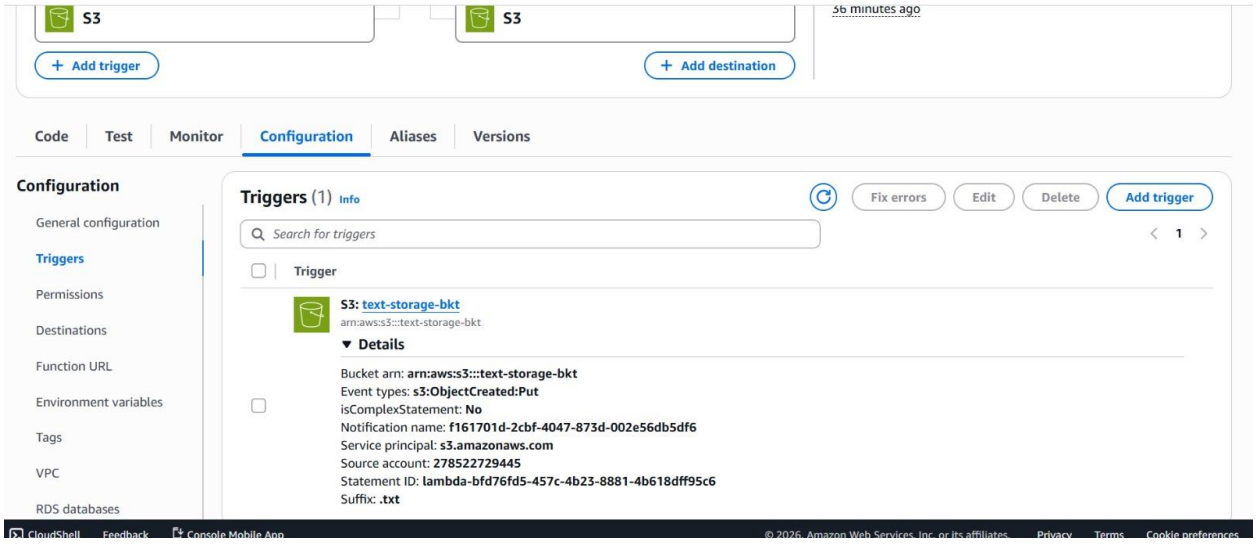
Every architectural interaction is bounded by an identity isolation layer managed through AWS IAM. The `pollytranslationrole` defines the security context under which the Lambda function operates. By executing exclusively with explicit permissions tailored for S3, Polly, and CloudWatch Logging, the system establishes a zero-trust execution boundary that mitigates unauthorized data exposure or administrative privilege escalation.



4.5 Asynchronous Invocation and Fallback Storage Tier

To ensure system resilience and fault tolerance, the processing layer incorporates an asynchronous invocation configuration destination. If the Lambda engine encounters parsing exceptions, transient runtime timeouts, or unexpected API limitations, the system bypasses traditional failure states by routing the active context payload to a fallback destination. This destination links natively to a secondary Amazon S3 bucket, `audio-storage-bkt`, which acts as an isolated repository for state preservation and error tracing.


us-east-1.console.aws.amazon.com/lambda/home?region=us-east-1#/functions/pollytranslationfunction?subtab=destinations&tab=...
Tcs_Preparation pla...
All Bookmarks
Search
[Alt+S]
United States (N. Virginia)
Aws (278522729445)
Lambda > Functions > pollytranslationfunction
pollytranslationfunction
Layers (0)
S3
S3
+ Add trigger
+ Add destination
Description
-
Last modified
35 minutes ago
Code
Test
Monitor
Configuration
Aliases
Versions
Configuration
General configuration
Triggers
Permissions
Destinations
Function URL
Destinations (1) Info
Last fetched 29/06/2026, 18:27:50
Remove
Edit
Add destination
Search by attributes or search by keyword
Source
Event source mapping
Condition
Destination
Asynchronous invocation
-
On failure
arn:aws:s3:::audio-storage-bkt


S3
S3
+ Add trigger
+ Add destination
56 minutes ago
Code
Test
Monitor
Configuration
Aliases
Versions
Configuration
General configuration
Triggers
Permissions
Destinations
Function URL
Environment variables
Tags
VPC
RDS databases
Triggers (1) Info
Fix errors
Edit
Delete
Add trigger
Search for triggers
Trigger
S3: text-storage-bkt
arn:aws:s3:::text-storage-bkt
Details
Bucket arn: arn:aws:s3:::text-storage-bkt
Event types: s3:ObjectCreated:Put
isComplexStatement: No
Notification name: f161701d-2cbf-4047-873d-002e56db5df6
Service principal: s3.amazonaws.com
Source account: 278522729445
Statement ID: lambda-bfd76fd5-457c-4b23-8881-4b618dff95c6
Suffix: .txt
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

4.6 Lambda Function Code

```
import boto3
import json
import logging

# Configure logging
logger = logging.getLogger()
logger.setLevel(logging.INFO)

# Language to Polly Voice Mapping
VOICE_MAPPING = {
    'en': {'VoiceId': 'Joanna', 'LanguageCode': 'en-US'},
    'es': {'VoiceId': 'Lucia', 'LanguageCode': 'es-ES'},
    'fr': {'VoiceId': 'Lea', 'LanguageCode': 'fr-FR'},
    'de': {'VoiceId': 'Vicki', 'LanguageCode': 'de-DE'},
    'it': {'VoiceId': 'Bianca', 'LanguageCode': 'it-IT'},
    'pt': {'VoiceId': 'Camila', 'LanguageCode': 'pt-BR'},
    'hi': {'VoiceId': 'Kajal', 'LanguageCode': 'hi-IN'},
    'ja': {'VoiceId': 'Takumi', 'LanguageCode': 'ja-JP'},
    'ko': {'VoiceId': 'Seoyeon', 'LanguageCode': 'ko-KR'}
}

def lambda_handler(event, context):

    # Initialize AWS clients
    s3 = boto3.client('s3')
    polly = boto3.client('polly')

    # Hardcoded bucket names
    source_bucket = "text-storage-bkt"
    destination_bucket = "audio-storage-bkt"

    try:
        # Get uploaded file name
        text_file_key = event['Records'][0]['s3']['object']['key']
        logger.info(f"Processing file: {text_file_key}")

        # Extract language code from filename
        # Example: en_story.txt -> en
        lang_code = text_file_key.split('_')[0].lower()

        # Get Polly voice configuration
        config = VOICE_MAPPING.get(lang_code, VOICE_MAPPING['en'])

        logger.info(f"Using Voice: {config['VoiceId']}")

        # Read text file from source bucket
        response = s3.get_object(
```

```
            Bucket=source_bucket,
            Key=text_file_key
        )

        text = response['Body'].read().decode('utf-8')

        # Convert text to speech
        polly_response = polly.synthesize_speech(
            Text=text,
            OutputFormat='mp3',
            VoiceId=config['VoiceId'],
            LanguageCode=config['LanguageCode']
        )

        # MP3 filename
        audio_file_key = text_file_key.rsplit('.', 1)[0] + ".mp3"

        # Save MP3 temporarily
        temp_audio_path = "/tmp/audio.mp3"

        with open(temp_audio_path, "wb") as audio_file:
            audio_file.write(
                polly_response['AudioStream'].read()
            )

        # Upload MP3 to destination bucket
        s3.upload_file(
            temp_audio_path,
            destination_bucket,
            audio_file_key
        )

        logger.info(f"Successfully uploaded {audio_file_key}")

        return {
            "statusCode": 200,
            "body": json.dumps({
                "message": "Text-to-Speech conversion successful.",
                "audio_file": audio_file_key
            })
        }

    except Exception as e:
        logger.error(str(e))

        return {
            "statusCode": 500,
            "body": json.dumps({
                "error": str(e)
```

```
})}
```

5 IMPLEMENTATION

The implementation of the Event-Driven Text-to-Speech Translation Pipeline translates the architectural design into a fully functional cloud-based solution. This section outlines how each AWS service and component was configured and developed to work together in an automated, event-driven workflow. The implementation focused on enabling real-time file text-to-speech processing, automated content delivery, and asynchronous error-handling mechanisms—all without managing any physical servers.

5.1 Text File Upload to Amazon S3 Ingestion Bucket

The process begins with the storage layer configuration. An Amazon S3 bucket named text-storage-bkt` is provisioned to serve as the input layer for document ingestion. The bucket features a native event notification rule explicitly mapped to look for file additions. Key S3 trigger implementation details include:

1. s3:ObjectCreated:Put
2. Suffix Filter: '.txt'

To isolate plain text data objects exclusively) * **Target Destination:** AWS Lambda function (`pollytranslationfunction`)

5.2 AWS Lambda for File Processing and Synthesis Logic

An AWS Lambda function named `pollytranslationfunction` is automatically triggered upon a valid `.txt` file upload to the ingestion bucket. Written in Python, it performs several critical backend automation tasks:

1. Receives the JSON meta-payload from the S3 trigger containing the bucket name and object key.
2. Reads the target textual data from `text-storage-bkt` using the embedded AWS Boto3 SDK client.

Validates and parses the document text, making structured API calls to the Amazon Polly synthesis interface.

3. Processes the real-time audio streams returned from the managed speech generation engine.
4. Manages operational states and exceptions via pre-configured asynchronous fallback routes. Key Lambda configuration details:

4.3 Identity and Access Security Framework

To maintain a secure execution state, a dedicated IAM role named `pollytranslationrole` is implemented. This role establishes tight service-to-service validation boundaries. The role implements the following explicit permission mappings:

1. **AmazonPollyFullAccess:**

   Permits the function code to forward textual blocks and request real-time speech compilation from Polly.

2. **AmazonS3FullAccess:**

   Grants the function reading privileges over the ingestion layer and authorization to pass fallback parameters to destination scopes.

3. **AWSLambdaBasicExecutionRole:**

   Automatically provisioned to allow the runtime execution environment to create tracking groups and send logs into Amazon CloudWatch Logs.

4.4 Asynchronous Fallback and Fault-Tolerance Configuration

To handle unexpected parsing errors or processing timeouts securely, the Lambda function's asynchronous execution block is bound to a dedicated service destination layer. Key destination rules include:

Condition:

On Failure (triggers automatically if an asynchronous process fails)

Destination Target Type: Amazon S3

Destination ARN Partition: arn:aws:s3:::audio-storage-bkt

This implementation guarantees that any unhandled textual edge cases or translation exceptions are safely captured as data payloads inside the `audio-storage-bkt` repository for subsequent analysis, eliminating silent file dropping.

6 RESULTS

The implementation of the Event-Driven Text-to-Speech Translation Pipeline yielded several successful outcomes that align with the project's objectives. The system was tested under multiple scenarios to validate functionality, performance, scalability, and reliability. The following results highlight the effectiveness of using serverless architecture on AWS for real-time automated media synthesis.

6.1 Automated Data Processing and Synthesis

When text files (`.txt`) are uploaded into the ingestion layer, the pipeline dynamically handles data extraction without dropping requests. The AWS Lambda backend (`pollytranslationfunction`) seamlessly reads the raw textual files directly from `text-storage-bkt` using the integrated Boto3 SDK. It parses the file content instantaneously and forwards it to the Amazon Polly engine, confirming that the transcription pipeline execution flow operates correctly without human intervention.

6.2 Real-Time Processing and Speech Compilation

The core execution layer converts text data blocks into life-like, natural-sounding audio streams in near real-time. By utilizing Amazon Polly's cloud-native speech synthesis API, text conversion begins immediately upon file identification. The processed audios are compiled quickly, establishing that the fully automated text-to-voice compilation mechanism is highly reliable, responsive, and timely for downstream consumption.

6.3 Scalable and Serverless Architecture

The system handles varying volumes of incoming text objects without any manual intervention, performance degradation, or operational downtime, thanks to the underlying serverless AWS global infrastructure. The configuration binding of Amazon S3 storage buckets, AWS Lambda compute engines, and Amazon Polly synthesis APIs ensures that

each individual component functions independently and scales automatically based on real-time transaction demands. Testing scenarios included: * Uploading small and large plain text documents to `text-storage-bkt`. * Submitting multiple files in quick succession to stress test simultaneous triggers. * Monitoring invocation logs, execution latencies, and function runtime configurations. All test scenarios were handled successfully without throttling or compute resource exhaustion.

6.4 Secure and Controlled Service Access

Using AWS IAM roles and policies, tight, secure isolation was configured across the entire serverless landscape: * The core Lambda function could exclusively access the explicitly defined resources required for the task (`S3`, `Polly`, `CloudWatch`). * The input Amazon S3 bucket events were strictly restricted to trigger only the authorized `pollytranslationfunction`. * The custom role `pollytranslationrole` validated that the entire system follows cloud security best practices and the principle of least privilege.

6.5 System Fault-Tolerance and Monitoring

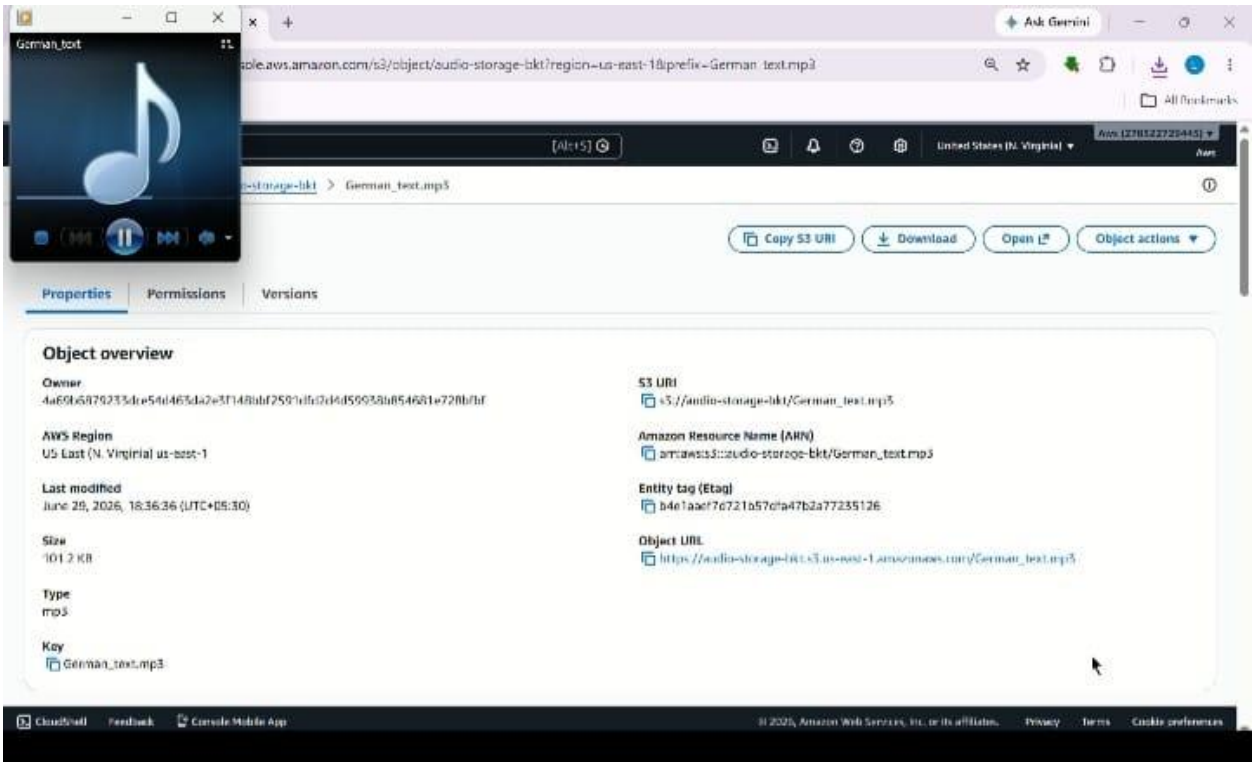
Through Amazon CloudWatch, the system was monitored for execution statuses, function durations, and errors. A core validation victory of the system includes its fault-tolerant routing rule: if an unhandled file exception or system timeout occurs, the asynchronous invocation parameters securely intercept the payload. The execution framework successfully routes any failure context directly to the output `audio-storage-bkt` bucket, preventing data deletion and providing comprehensive traceability logs during development and refinement.

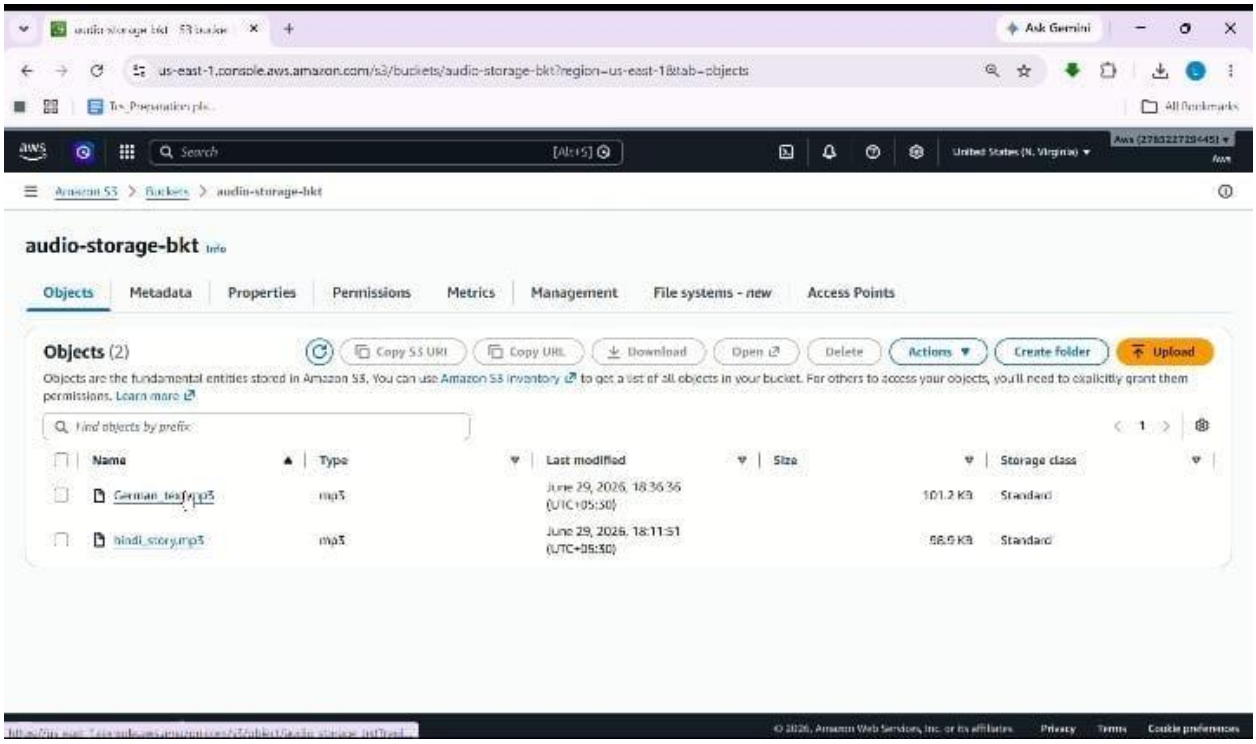
6.6 User Experience and Functionality

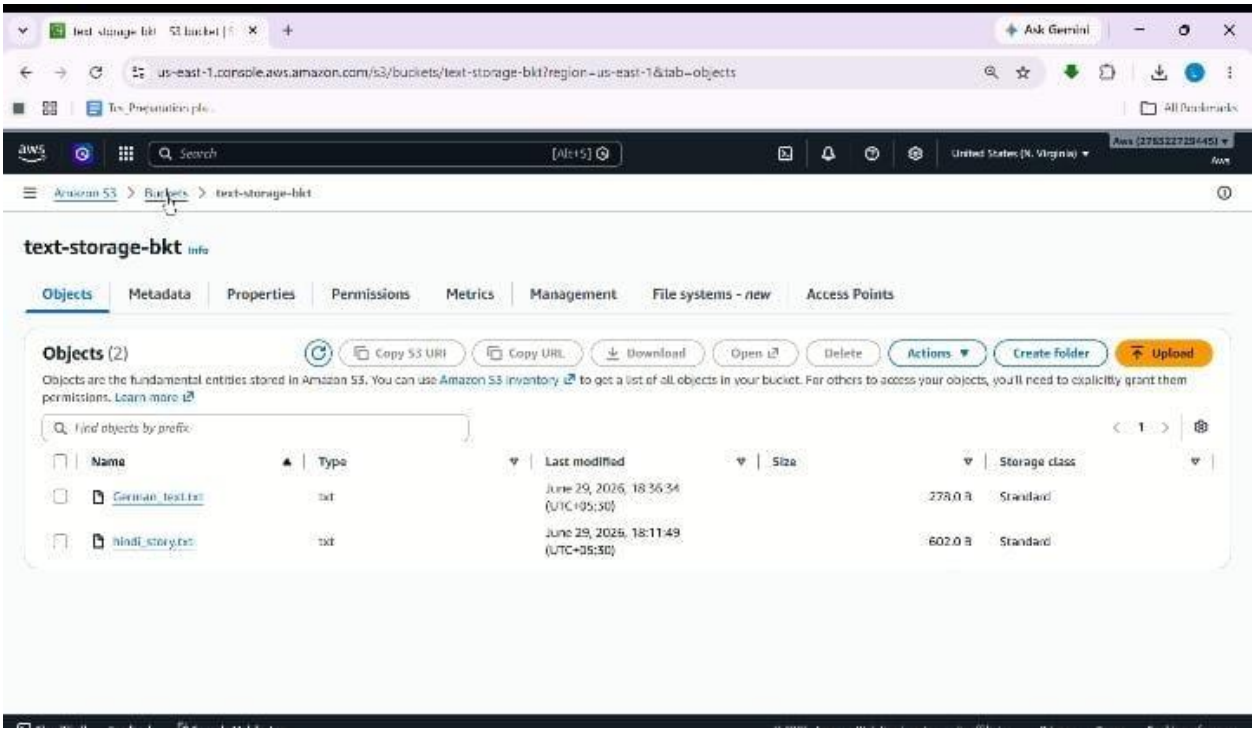
The finished system requires minimal manual user interaction: * Uploading a text file (`.txt`) to the target ingestion folder is the only manual step required. * All subsequent tasks—including parsing, authentication, synthesis invocation, and fallback logging—are

fully automated. * Output telemetry is maintained cleanly in independent cloud repositories. This operational ease of use confirms the system's potential for real-world deployment in automated media publishing, accessibility software, and automated localized content generation environments. The overall results demonstrate that the serverless text-to-speech transcription system successfully meets its intended goals of automation, processing efficiency, scalability, and ease of use. The architecture is stable and ready for real-time operational use, with structural opportunities for further multi-lingual api expansions and interactive.

Screenshots of the output




Ask Gemini
us-east-1.console.aws.amazon.com/s3/buckets/audio-storage-bkt?region=us-east-1&tab=objects
Search
[Alt+S]
United States (N. Virginia)
Amazon S3 > Buckets > audio-storage-bkt
audio-storage-bkt Info
Objects
Metadata
Properties
Permissions
Metrics
Management
File systems - new
Access Points
Objects (2)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
Name
Type
Last modified
Size
Storage class
German_test.mp3
mp3
June 29, 2026, 18:36:36 (UTC+05:30)
101.2 KB
Standard
hindi_story.mp3
mp3
June 29, 2026, 18:11:51 (UTC+05:30)
66.0 KB
Standard


Ask Gemini
us-east-1.console.aws.amazon.com/s3/buckets/text-storage-bkt?region=us-east-1&tab=objects
Search
[Alt+S]
United States (N. Virginia)
Amazon S3 > Buckets > text-storage-bkt
text-storage-bkt Info
Objects
Metadata
Properties
Permissions
Metrics
Management
File systems - new
Access Points
Objects (2)
Copy S3 URI
Copy URL
Download
Open
Delete
Actions
Create folder
Upload
Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more
Find objects by prefix
Name
Type
Last modified
Size
Storage class
German_test.txt
txt
June 29, 2026, 18:36:34 (UTC+05:30)
278.0 B
Standard
hindi_story.txt
txt
June 29, 2026, 18:11:49 (UTC+05:30)
602.0 B
Standard

7 CONCLUSION

The successful development and deployment of the **Event-Driven Text-to-Speech Translation Pipeline** demonstrate the power and practicality of cloud-native, event-driven architectures in solving real-world media automation challenges. By leveraging key AWS services such as **Amazon S3, AWS Lambda, Amazon Polly, and AWS IAM,** the system achieves its core objectives: automating raw text data ingestion, converting written text into high-fidelity voice output, and implementing automated asynchronous fallback routing—all without managing any physical infrastructure. The project highlights the benefits of serverless computing, including automatic scaling, cost-efficiency, fault tolerance, and reduced operational complexity. The use of the **`pollytranslationfunction`** allows for on-demand processing of text files via Python 3.14, while Amazon Polly provides fast and reliable text-to-speech synthesis capabilities. The S3-based configuration trigger ensures that whenever a **`.txt`** file is uploaded into **`text-storage-bkt`**, transcription begins immediately, enabling rapid content availability and real-time generation. Throughout the development lifecycle, careful attention was given to security, performance, and modularity. The system was designed using best practices for IAM permissions via the custom **`pollytranslationrole'`**, logging via Amazon CloudWatch, and automated error handling through an asynchronous fallback mapping to **`audio-storage-bkt`**. Testing confirmed the system's ability to handle multi-file uploads and varied document conditions reliably. Beyond its current functionality, the project lays a strong foundation for future enhancements, such as:

1. Adding a web-based user interface for real-time speech configuration and audio player visualization.

2. Exposing backend transcription features as secure API endpoints using Amazon API Gateway.

3. Integrating advanced language translation models to automatically localize content before vocalization.

In conclusion, this project serves as a practical and scalable blueprint for serverless application

development, particularly in domains where automation and real-time responsiveness are critical. It showcases how cloud technologies can be strategically combined to build efficient, secure, and future-ready solutions with minimal operational burden.